Where are we

- Java interfaces
  - Introduction
  - Syntax
  - UML notation
  - Multi-inheritance of interfaces
  - (Some) Java pre-defined interfaces
  - Design pattern: Strategy

1

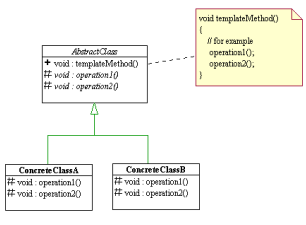
A Quicksort class as the Template Method abstract class

```
@SuppressWarnings("unchecked")

abstract class Quicksort {
   private Comparable[] data;

    public final void quicksort(Comparable[] d) {
      data = d;
      quicksort(0, data.length-1);
   }

    /** quicksort method as the template method */
    private final void quicksort(int left, int right) {
        if (right <= left) return;
        int i = partition(left, right);
        quicksort(left, i-1);
        quicksort(i+1, right);
   }
```



2

Java pre-defined interfaces

```
Interface Comparable<T> {
    /** Answer a negative integer if the receiver is lower than the
    argument obj;
       answer 0 if both receiver and obj are equal;
       answer a positive integer is receiver is greater than obj
    */
    int compareTo(T obj);
}
```

```
@Deprecated
interface Comparable {
    /** Answer a negative integer if the receiver is
    lower than the argument obj; answer 0 if both
    receiver and obj are equal; answer a positive
    integer is receiver.is greater than obj
    */
    int compareTo(Object obj);
}
```

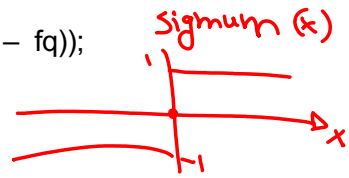
```
Fraction f, g;
//...
IO.println ( f.compareTo (g));
```


Example

```
/** @version 3.0
*/
class Fraction implements Comparable<Fraction> {
        private int num, den;
        // …

        public int compareTo (Fraction f) {
                double  q = (double) num / (double) den;  //receiver
                double  fq = (double) f.num /(double) f.den ;  //f

                return ((int) Math.signum(q –  fq));
        }
}
```



Example

```
public class Main {
    void main() {
        Fraction [] a = {new Fraction(1, 7),
                         new Fraction(5, 2),
                         new Fraction(1, 2) };

       ProbQuickSort<Fraction> pqs = new ProbQuickSort<>();

        pqs.quicksort(a);
       for(int i=0; i< a.length; i++ )
                IO.println (a[i]);
   }
}
```

Now we can use quicksort to sort **Fraction**

Pre-defined Java interfaces

```
interface Cloneable {
}
```

No methods
a markup interface

Main methods of abstract superclass Object



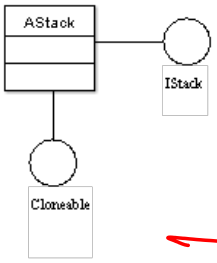
- protected Object clone()
  throws CloneNotSupportedException
- public boolean equals(Ob
- protected void finalize() t
- public final Class<?> getClass()
- public String toString()

inherited by all Java classes

by default, all classes throw an exception

7

Example




```
Interface IStack {
    //…
}

class AStack implements IStack, Cloneable {
    //...
    public AStack() {
        this(DEFAULT_SIZE);
    }
    //...
    public Object clone() {
        // Override to avoid exception

        // Create a new stack with
        // the same elements of this
    }
}
```


8

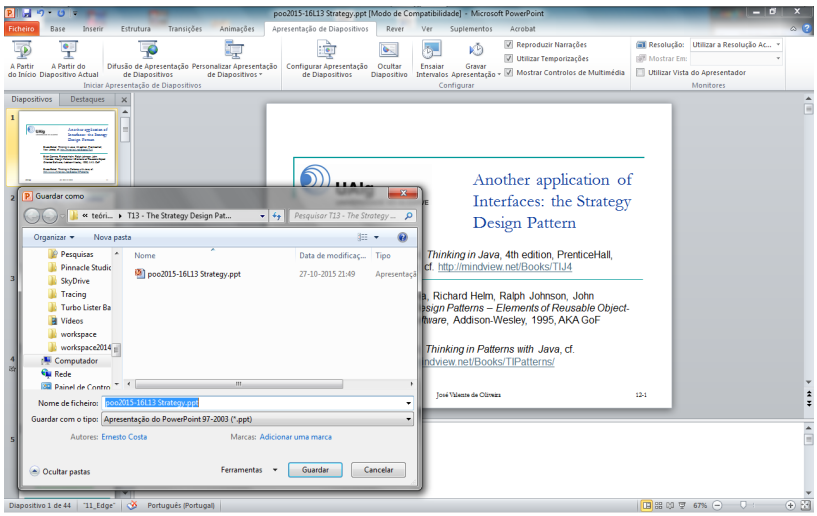
UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Another application of Interfaces: The Strategy Design Pattern

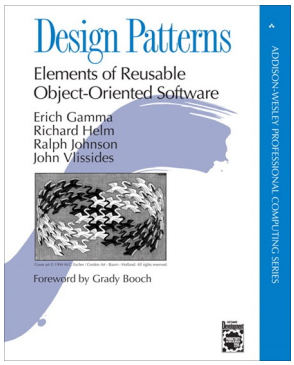
Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, *Design Patterns – Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995, AKA GoF

Bruce Eckel, *Thinking in Patterns with Java*, cf. http://www.mindview.net/Books/TIPatterns/

The problem



Strategy – intent

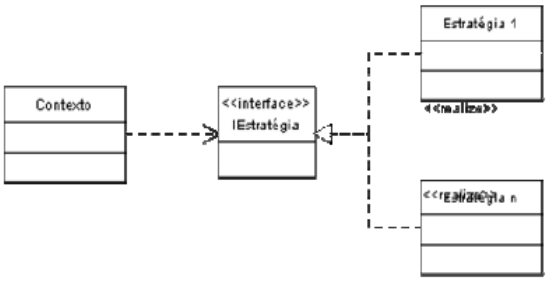


- Define a family of algorithms, encapsulate each one, and make them interchangeable
- Capture the abstraction in an interface that can be implemented by independent classes

Strategy – usage

- Use this pattern when an object should be parametrized with one of several algorithms that can be represented by a single interface
- Strategy allows one of a family of algorithms to be selected on-the-fly at runtime.

Strategy - structure



13

Strategy – java example

```
public abstract class Quicksort  // …
   protected final boolean less(int i, int j) {
            T di = data.get(i);
            T dj = data.get(j);
            return (di.compareTo(dj) < 0);
    }
}
```

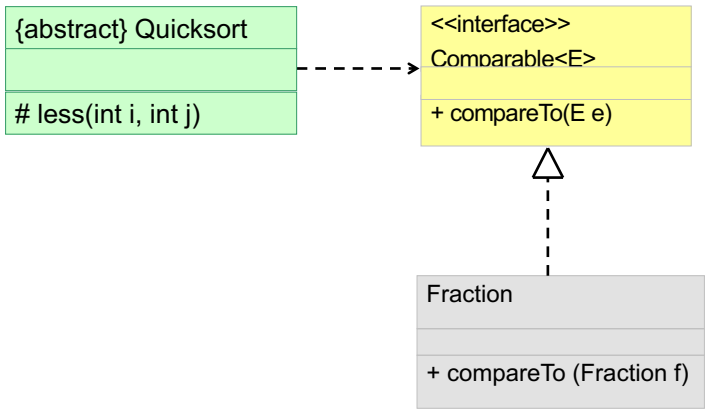
```
interface Comparable<E> {
        int compareTo(E e);
}
```

```
class Fraction implements Comparable<Fraction> {
    // …
    public int compareTo (Fraction f) {
            double  q = (double) num / (double) den ;
            double  fq = (double) f.num /(double) f.den;

            return ((int) Math.ceil(q –  fq));
}}
```


14

The same example in UML

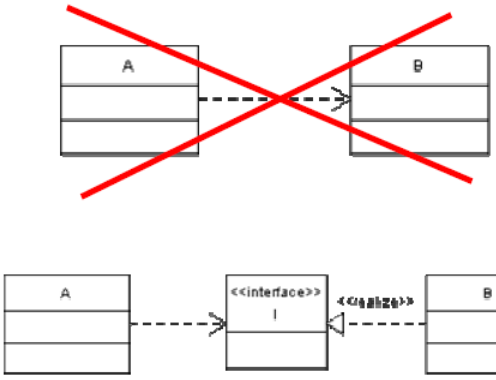


Strategy – Brief discussion

- The usage of the Strategy design pattern avoids the employment of conditional control structures that would make code hard to read and to maintain
- Client needs to be aware of the different available strategies
- In some cases, both Strategy or TemplateMethod can be used

  Typically, Strategy is more flexible

Program to an interface not for an implementation



17

Check point:

We found the code in this and in the next slide in the net…

```
public class Main {
    void main() throws FileNotFoundException {
                String s = "a String";

                FileDataHandler.write("test", s, "html");
                FileDataHandler.write("test", s, "xml");
    }
}
```

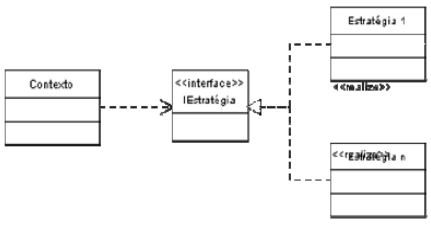

18

Check point: how to improve the Quality of this code?

```
class FileDataHandler {
    public static void write(String filename, String text, String selector)
                                throws FileNotFoundException {
        filename += '.' + selector;
        PrintWriter out = new PrintWriter(new File(filename).getAbsoluteFile());
        if (selector.compareToIgnoreCase("html")==0)
                out.print("<div><h1> " + text + "</h1></div>");
        else if (selector.compareToIgnoreCase("xml")==0)
                out.print("<?xml version="+""+"1.0"+""+" encoding="+""+"iso-
    8859-1"+""+"?> <data>" + text + "</data>");
                    else {
                            System.err.println("Invalid Selector");
                            System.exit(1);
                    }
    }
}
```

write("test", s, "html");

Recall the structure of Strategy




```
interface IDisplayContents{
    String display(String text);
    String fileExtension();
}
```

Applying the Strategy Design Pattern: 2 – Define the Context class

```
class FileDataHandler {
    public static void write(String filename, String text, IDisplayContents i)
                                throws FileNotFoundException {
        filename += i.fileExtension();
        PrintWriter out = new PrintWriter(new File(filename).getAbsoluteFile());
        out.print(i.display(text));
        out.close();
    }
}
```


The new FileDataHandler is now much more clear than the original

```
class FileDataHandler {
    public static void write(String filename, String text, String selector)
                                throws FileNotFoundException {
        filename += '.' + selector;
        PrintWriter out = new PrintWriter(new File(filename).getAbsoluteFile());
        if (selector.compareToIgnoreCase("ht...")==0)
                out.print("<div><h1>" + text + "</h1></div>");
        else if (selector.compareToIgnoreCase("xml")==0)
                out.print("<?xml version="+""+"1.0"+""+" encoding="+""+"iso-
    8859-1"+""+"?> <data>" + text + "</data>");
                    else {
                            System.err.println("Invalid Selector");
                            System.exit(1);
                        }
    }
}
```

Applying the Strategy Design Pattern:
3 – Write the concrete class that implements the Strategy interface

```
class StrategyHTML implements IDisplayContents {
   // return contents of data file as HTML
   public String display(String text){
     return "<div><h1>Contents of data file, formatted as HTML</h1><p>" +
            text + "</p></div>";
   }

   public String fileExtension() { return ".html";}
}


// define 'StrategyXML' class
class StrategyXML implements IDisplayContents {
   // return contents of data file as XML
   public String display(String text) {
     return "<?xml version="+""+"1.0"+""+" encoding="+""+"iso-8859-1"+""+"?>
     <data><filedata>" + text + "</filedata></data>";
   }

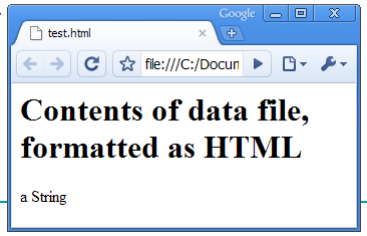
   public String fileExtension() { return ".xml";}
}
```

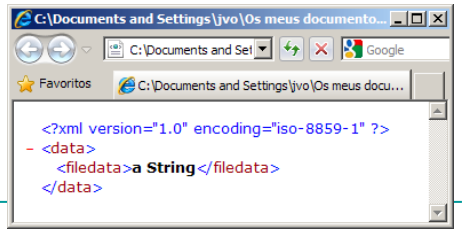

23

Applying the Strategy Design Pattern:
4 – Let the client decide on the strategy to use.

```
public class Main {
    void main() throws FileNotFoundException {
            String s = "aString";
            IDisplayContents d = new StrategyHTML();
            FileDataHandler.write("test", s, d);

            d = new StrategyXML();
            FileDataHandler.write("test", s, d);
    }
}
```





24

Summary

- Java interfaces
  - An introduction
  - Syntax
  - UML notation
  - Multi-inheritance of interfaces
  - (Some) Java pre-defined interfaces
  - Design pattern: Strategy